

BUCHAREST UNIVERSITY OF ECONOMIC STUDIES

Faculty of Business Administration Teaching in Foreign Languages (FABIZ)

**UNDERSTANDING WHY LLOYD'S ALGORITHM IN K-MEANS
CLUSTERING
FREQUENTLY STALLS AT SUBOPTIMAL CLUSTER SOLUTIONS:
A CASE STUDY IN CUSTOMER SEGMENTATION**

Submitted by:

RIFAT RAYHAN ROMY

Supervised by:

Professor Adriana Mihnea

Bucharest, Romania

2026

Abstract

This thesis starts with a simple but unsettling observation: Lloyd's algorithm, the fundamental iterative process at the heart of nearly every instance of K-Means executed in practice for business, research, or industry, always converges to a solution that is suboptimal. It always terminates. It always looks like it has arrived somewhere. But in most cases, it ends up at a suboptimal partition. In other words, there's no sign that anything is wrong. This phenomenon is called algorithmic stalling. Of course, the concept itself isn't novel. It's been known for decades that K-Means seeks to minimize a non-convex criterion, and optimization of non-convex criteria typically converges only to a local minimum. But the scope of the problem, and its implications in the practical case of analyzing transactions at a large supermarket chain, remain poorly understood – which is why this thesis was written. There's a difference between a demonstration of the existence of the problem on toy data sets with clean Gaussian cluster separation and demonstrating it on real ten-dimensional data generated from point-of-sale behavior data from an actual supermarket chain in Europe.

The data behind this study were gathered at one supermarket in Southern Poland, wherein the two researchers Antczak and Weron (2019) observed 162,926 checkout events in three seasonal patterns: before-Christmas holiday in December 2017, winter period in February 2019, and spring-Easter time in March-April 2019. Instead of creating clusters from the raw checkout events, in which there is no true value of K , I generated ten Data Blocks with Gaussian distribution to mimic the statistical properties of the actual checkout event data but with a cluster structure of $K_{\text{TRUE}}=7$. Through such a process, both reality and ground truth were created, making it possible to measure stalling accurately.

The experiments are designed such that Lloyd's algorithm and K-Means++ are each run for 1,000 times per condition with varying number of clusters, k , from 2 up to 12, on all ten Gaussian Data Blocks. I propose the use of Mean Centroid Displacement (MCD) as the performance metric, where the MCD measures the total distance traveled by the centroids per iteration, not the within-cluster sum of squares (WCSS), the decline of which with respect to the incrementing k value guarantees the inherent lack of reliability when comparing WCSS values with different k values. From the MCD, I derive the concept of Lloyd Bias, a non-dimensional ratio of mean displacement against the actual k value.

The conclusion is clear. In 99.9% of the single run instances, Lloyd's algorithm gets stalled at $k = 7$. Lloyd Bias gives a consistent and smooth monotonous function graph ranging from 1.92 at $k = 2$ down to 0.975 at $k = 12$. Initialization based on K-Means++, which has been recommended by many researchers as an improvement from naive random seeding, has proven effective in reducing the centroid displacement variance by about 2–6%, but makes no difference to the problem of stalling at $k = 7$, where it reduces stalling probability by just 0.1%.

The findings carry direct practical consequences. A client classification procedure that makes use of a single iteration of K-Means without considering the fact that by default, scikit-learn employs ten iterations will almost certainly result in an unstable, non-reproducible, and possibly misleading solution. This thesis presents a thorough analysis of the issue at hand along with practical guidelines for detecting stalling in K-Means based algorithms.

Keywords:

Lloyd's algorithm, K-Means clustering, local minima, algorithmic stalling, customer segmentation, Monte Carlo simulation, Mean Centroid Displacement, K-Means++, POS data, retail analytics, Polish supermarket, convergence metrics, Lloyd Bias

Table of Contents

Abstract	2
Chapter 1: Introduction	4
1.1 The Failure That Looks Like Success	4
1.2 Research Context: A Real European Supermarket	5
1.3 What Stalling Actually Means	6
1.4 Research Objectives	7
1.5 Contributions to Knowledge	7
1.6 How This Thesis Is Organized	8
Chapter 2: Literature Review	9
2.1 Origins and Formulation of K-Means	9
2.2 Non-Convexity and the Local Minima Problem	10
2.3 Initialization Strategies	11
2.4 Convergence Theory and the Role of Dimensionality	14
2.5 Stalling in Customer Segmentation Applications	15
2.6 Evaluation Metrics for Clustering Quality	16
2.7 Mean Centroid Displacement as a Novel Convergence Metric	18
2.8 Multi-Start and Ensemble Approaches	19
2.9 Retail Analytics and the K-Means Segmentation Paradigm	20
2.10 Summary and Research Gap	21
Chapter 3: Data, Feature Engineering, and Experimental Design	22
3.1 The Polish Supermarket POS Dataset	22
3.2 Feature Engineering Pipeline	23

3.3 Gaussian Data Block Construction	25
3.4 Experimental Design	26
Chapter 4: Methodology	28
4.1 Implementation of Lloyd's Algorithm	28
4.2 Mean Centroid Displacement (MCD)	29
4.3 Lloyd Bias	30
4.4 Stalling Operationalization	31
4.5 Solution Consistency: Cross-Run ARI	31
4.6 Software Environment	32
Chapter 5: Results	33
5.1 Reference MCD at $k_{\text{true}} = 7$	33
5.2 Lloyd Bias Across k Values	33
5.3 Stall Rates	34
5.4 Solution Consistency: Cross-Run ARI	35
5.5 MCD Coefficient of Variation	36
5.6 Internal Validation Metrics	36
Chapter 6: Discussion	37
6.1 The Severity of Stalling in Practice	37
6.2 The Lloyd Bias Framework as a Diagnostic Tool	38
6.3 The Limited Benefit of K-Means++ Revisited	39
6.4 Seasonal Variation in Stalling Behavior	40
6.5 Practical Recommendations for Practitioners	40
6.6 Limitations and Boundary Conditions	42

Chapter 7: Conclusion	43
References	46
Appendix A: Consolidated Quantitative Findings	51
Appendix B: Experiment Configuration Summary	52

DECLARAȚIE / STATEMENT

Subsemnatul/a, **RIFAT RAYHAN ROMY**, student/ă în anul 2,
I, the undersigned, *student in the year of studies*
 an universitar 2025/2026, **absolvent/ă** al/a programului de studii universitare de masterat
academic year *graduate of the master's degree program in*
 Analize de Afaceri / Business Analytics, din cadrul Facultății
within the School of

Administrarea Afacerilor cu predare în limbi străine, declar că am utilizat instrumente de
Business Administration in foreign languages, hereby declare that I have used generative
 inteligență artificială generativă (IAG) în procesul de elaborare acestei lucrări de disertație cu
 titlul:

artificial intelligence tools (GAI) in the process of preparing this dissertation paper titled
 Understanding Why Lloyd's Algorithm in K-Means Clustering Frequently Stalls at Suboptimal
 Cluster Solutions: A Case Study in Customer Segmentation
 exclusiv în scopurile descrise mai jos, cu respectarea principiilor prevăzute în Ghidul de utilizare a
exclusively for the purposes described below, in compliance with the principles set forth in the
 instrumentelor de Inteligență Artificială Generativă (IAG) în cadrul Academiei de Studii
the Guide for the Use of Generative Artificial Intelligence (GAI) Tools at the Bucharest University
 Economice din București.
of Economic Studies

Instrumentul(le) utilizat(e) / *Tool(s) used*: Claude (Anthropic), ChatGpt, Perplexity, Gemini
 Scopul utilizării: (ex. traducere, revizuire lingvistică, generare de idei) – (Studentul/absolventul
 detaliază modul în care a fost utilizat IAG-ul și partea/părțile din lucrare în care acesta a fost
 utilizat) / *Purpose of use: (e.g., translation, linguistic review, idea generation) – (The*
student/graduate details how the GAI was used and the section(s) of the paper in which it was
used: Linguistic review and writing assistance: GAI was used to improve the clarity and fluency of
 written sections throughout the thesis, including the Abstract, Introduction (Chapter 1), Literature
 Review (Chapter 2), Discussion (Chapter 6), and Conclusion (Chapter 7). The tool was also used
 for idea generation and structuring of arguments, particularly in formulating the Lloyd Bias
 framework and practical recommendations (Section 6.5). , data completă a interacțiunii (anul, luna
 și ziua) / *full date of interaction (year, month, and day)*: 2026, January–May
 Sunt conștient/ă că, în cazul constatării falsului, voi suporta rigorile legii pentru săvârșirea
I am aware that, in the event of forgery being detected, I will face the full force of the law for
 infracțiunilor comise (fals, uz de fals, fals material în înscrisuri oficiale), conform Art. 326 din
the crimes committed (forgery, use of forged documents, material forgery in official documents),
 in

Codul penal, cu modificările și completările ulterioare.
accordance with Article 326 of the Criminal Code, as amended and supplemented.

A handwritten signature in black ink, appearing to read 'Rifat', written in a cursive style.

Data/ Date: 04/06/2026

Semnătură / Signature,

Chapter 1: Introduction

1.1 The Failure That Looks Like Success

Then there is a type of failure in algorithms which is even more perilous than the typical kind of failure which leads to the crash or exception of programs. This failure happens to present itself externally as success in the program. This is evident whenever Lloyd's algorithm terminates successfully for convergence, and when this happens, it never fails to return an array of cluster assignments which is neatly formatted, along with centroid coordinates which have achieved the optimal value within a local minimum. The algorithm has succeeded in meeting all its expectations.

It has succeeded except in the most crucial task: finding the optimal solution.

This is what I sought to investigate, and it proves to be far more serious in reality than one might gather from theoretical literature. K-Means clustering, or rather, the collection of algorithms using the iterated assignment/centroid update algorithm pioneered by Lloyd in 1982, is one of the most commonly used forms of unsupervised learning. Retail firms rely extensively on the method to classify their customers into actionable segments characterized by their shopping habits, frequency of purchases, composition of the basket, timing, amount spent, and preferred modes of payment. This is not merely an academic matter. Marketing teams design campaigns in light of the segmentation. Logistics teams plan inventory delivery considering the same. And so forth. The consequences of faulty segmentation in such cases are dire, but no one will be aware of that fact at any stage of the process.

The randomness I speak of is inherent in the structure of the system, not a coincidence. Lloyd's clustering starts by selecting a random number of cluster centroids, which are then iteratively updated as each data point is assigned to the closest centroid; this process continues until there is no change in the positions of centroids, calculated as the meaning of all points assigned to them. Coordinate descent is used to find a local minimum for an inherently non-convex function in this instance. Since it can be assured that a non-convex coordinate descent will arrive at a local minimum, there is no way of reaching the global minimum.

This research focuses on quantifying such issues within the context of actual data obtained from a supermarket located in South Poland. Specifically, this thesis analyzes transaction data containing 162,926 Point-of-Sale transactions using real transaction distributions. The concise

results of my investigation are summarized as follows. For $k=7$ clusters, Lloyd's algorithm fails to converge in 99.9% of individual executions, while any two distinct executions are capable of reaching consensus on cluster membership just slightly above the probability expected by random chance. An explanation for these findings along with a comprehensive discussion will be provided in the rest of this thesis.

The real contribution of this research lies in the quantification of exactly how problematic the convergence of K-Means truly is within a commercially relevant scenario, as opposed to just adding yet another example of why K-Means fails to converge. The contribution lies in presenting measures of this problem (Mean Centroid Displacement and Lloyd Bias), which have previously been ignored by the literature, as well as providing actionable solutions for this issue.

1.2 Research Context: A Real European Supermarket

The key data used for this research was sourced from a single supermarket belonging to a large European retail chain, operating in Southern Poland. The data were provided by Antczak and Weron (2019) and published through MDPI Data. They refer to 162,926 Point-of-Sale transactions made across three different seasons of trading: pre-Christmas in December 2017, normal winter season in February 2019, and the spring-Easter season from March to April 2019. The data used are not artificial data, not benchmark data, and not a fabricated set just to show the capabilities of clustering techniques.

This particular choice was not accidental but purposeful, and it has consequences for the analysis results. In the first place, the seasonality pattern itself is interesting from an analytical standpoint, since December is truly an abnormal month to do grocery shopping – higher basket sizes, unique products because of the Christmas preparations, and condensed holiday shopping in one week. February is the nearest month to normal groceries throughout the year. The months March-April bring additional features related to Easter and an increase in the purchase of fresh produce, resulting in inflation of family-sized baskets, halfway between those of December and those of February. This variation in the geometry of the feature space is crucial in the context of the stalling problem, because it defines the way the WCSS surface looks like.

Out of the pure log data, ten features have been created which cover unique aspects of user behavior during the checkout process, including overall transaction time, mean time spent idle

between scanning articles, number of items bought, overall expenditure in Polish Zloty, mean cost of items purchased, hour of the day, day of the week, indicator for weekend, and two payment method flags (whether card or cash was used). Altogether, this results in a feature space of ten dimensions, which belongs to what may be considered an intermediate dimensionality domain, sufficiently complex that initialization is a problem, but not too complex that results from theoretical worst-cases are directly applicable.

One key methodological choice I made was not to cluster on the actual data itself. If I had done so, there would be no way for me to test if the clustering done by Lloyd's algorithm, which stalls halfway, is incorrect. For that reason, I created ten Gaussian Data Blocks datasets with characteristics designed to replicate those found in the real-world sample data (such as mean, standard deviation, range, and coefficient of variation). However, in contrast with the real-world dataset, each of the ten created datasets has the built-in ground truth structure of $K_TRUE=7$.

1.3 What Stalling Actually Means

In common parlance, "stalling" implies indecision, delay, or cessation before reaching one's destination. For me, as used elsewhere throughout this thesis, it carries a more precise meaning. The algorithm reaches its destination swiftly, yet again and again, lands at an erroneous destination.

For example, when I ran Lloyd's algorithm 1,000 times, starting each time from a different randomly generated initialization point, for the same Gaussian Data Block, and set k equal to seven (the appropriate number of clusters), my expectation was that, if all 1,000 runs yielded identical partitions, then Lloyd's algorithm is dependable in this scenario: that is, the global optimum predominates and lies in the attraction basin for any possible random initialization. However, instead, my Adjusted Rand Index between two randomly selected runs out of 1,000 was 0.49. That is, there is very little overlap in the clusters found by two randomly selected runs.

Now consider what that entails in practical business applications. On one Monday, an analyst uses K-Means with seed 42 and creates clusters of clients. The following day, his/her colleague repeats the exact procedure, using seed 77 instead. As many as 25% to 40% of the customers wind up in different clusters. All the results are equally legitimate. Both analyses have valid WCSS scores, and neither shows any signs of being erroneous. The computer just makes a different arbitrary decision when choosing between two similar solutions.

At the same time, fully 99.9% of these 1,000 iterations can be labeled "stalled" using my within-condition criteria: their Mean Centroid Displacement value is higher than 1.05 times that of the optimal iteration. In other words, essentially all of these iterations perform worse than the best iteration seen, and the remaining 0.1% come very close. This is not occasional bad luck; the algorithm almost always performs poorly.

This is precisely why stalling poses such an interesting issue for empirical research. This is something the theory has always indicated us could occur. It is the discovery of this thesis that this behavior occurs all the time in the context of actual retail data and that the common mitigating techniques are significantly less effective than commonly believed.

1.4 Research Objectives

This thesis pursues four primary research objectives:

RO1: Quantification of stalling frequency and magnitude.

The present thesis will address four key areas of study:

RO1: Measurement of stalling frequency and intensity.

To determine the frequency and intensity of stalling Lloyd's algorithm exhibits across the full spectrum of k values ranging from $k=2$ to $k=12$, using a Monte Carlo simulation approach involving 440,000 Lloyd's algorithm iterations on ten sets of synthetic Gaussian Data Blocks generated based on the Polish supermarket point-of-sale data set.

RO2: Description of the Lloyd Bias phenomenon.

To establish and explain the connection between number of clusters and Lloyd Bias understood as the ratio between mean cluster center displacement at k and at true number of clusters serving as a new convergence diagnostic technique exposing geometry of stalling throughout the k space.

RO3: Validation of K-Means++ as a potential remedy against stalling.

To assess whether K-Means++ initialization algorithm helps mitigate Lloyd's stalling issue or only shifts it, conducting K-Means++ experiments in exactly the same conditions as Lloyd's random initialization, comparing stalling rate, mean displacement variance and solution variability.

RO4: Guidelines for customer segmentation specialists.

To provide data-driven advice to retail data scientists concerning circumstances under which single iteration K-Means can be safely used or avoided, and what procedures ensure better clustering results.

1.5 Contributions to Knowledge

Firstly, the thesis contributes the definition of Mean Centroid Displacement as a convergence criterion. Currently, within-cluster sum of squares is the dominant method used to measure convergence quality in K-Means clustering researches, yet this statistic tends to decrease monotonically with an increase in k and thus can hardly be utilized to compare convergence quality between cluster numbers. As MCD measures centroid displacement per iteration, it provides unbiased by k convergence metric and directly illustrates how hard the algorithm works in the optimization of information of a completely different nature.

Secondly, the thesis offers Lloyd Bias – a dimensional-less ratio diagnostic method that uses the MCD of the correct number of clusters as a denominator to assess the entire spectrum of k with respect to convergence effort. The results of Lloyd Bias analysis show a clear geometric pattern that was never observed before (monotonically decreasing from 1.92 to 0.975 when changing from $k = 2$ to $k = 12$) and that might have a lot of practical applications in k selection process and stall assessment.

The third contribution is that of the benchmarks themselves, the first major analysis of Lloyd's stalling using Monte Carlo methods on actual transaction data from Europe, complete with stall statistics, inter-run ARI numbers, and displacement variability estimates that serve as a basis of comparison and expectation for commercial use. The methods used to create the Gaussian Data Blocks and conduct the Monte Carlo analysis are sufficiently described for reproducibility elsewhere.

1.6 How This Thesis Is Organized

Chapter 2 is dedicated to an overview of the literature on K-Means, Lloyd's algorithm, initialization techniques, convergence theory, and applications to retail segmentation, with special attention paid to the publications that appeared within the last five years. Chapter 3 provides an overview of the Polish POS data, the feature engineering process, and the Gaussian Data Blocks construction technique. Chapter 4 introduces the experiment design framework, including the custom algorithm implementation, MCD and Lloyd Bias metrics, stall definition, and software configuration. Chapter 5 provides experimental results for all k values and configurations. Chapter 6 discusses results in the context of the literature and presents practical suggestions. Chapter 7 summarizes the main results, limitations, and future work directions.

Chapter 2: Literature Review

2.1 Origins and Formulation of K-Means Clustering

Indeed, it seems pertinent, before proceeding further, to reflect on how old K-Means is and how amazing it is that an algorithm conceived in mid-twentieth-century mathematics is still the most prevalent clustering technique in contemporary commercial data science. The concept of partitioning based on centroids can be traced back to Steinhaus (1956), where the problem of partitioning spatial points based on centers in Polish mathematical statistics was proposed. The name K-Means and the online updating algorithm were first introduced by MacQueen (1967). However, the iterative algorithm that is associated with `sklearn.cluster.KMeans` in contemporary data science was formulated by Stuart Lloyd in his Bell Laboratories Technical Report from 1957, which he saw only published in 1982 – hence the twenty-five-year delay explaining why the algorithm is both referred to as K-Means in software and Lloyd's in academic literature.

By definition, for a dataset $X = \{x_1, x_2, \dots, x_n\} \subseteq \mathbb{R}^d$ and a number of clusters to find k , Lloyd's algorithm finds the assignment that minimizes the within-cluster sum of squared errors (WCSS), defined by the equation: $WCSS(C) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$. This cost function is non-convex in the space of assignments and centroids together. It is guaranteed that convergence will take place in finite iterations, and this guarantee is definite and real. The guarantee of optimality that this solution will be global, on the other hand, is not.

The continued dominance of the algorithm in question has been well-captured through the efforts of Ikotun et al. (2023), who, in their systematic review of K-Means literature over two decades, discovered that the algorithm was featured in more than 6,200 articles indexed in Scopus between the years 2000 and 2021. It is clear, then, that the two factors underpinning the persistence of this approach include explainability and efficiency. This is especially relevant in the current environment where machine learning predictions are coming under increasing scrutiny from regulatory bodies, from the EU AI Act to specific governance schemes, making both factors of greater worth than ever before.

Recent theory has clarified conditions under which even this weak condition can be relied upon. In their recent study, Choo and Vempala (2024) gave results concerning the expected approximation ratio of Lloyd's algorithm using a Gaussian mixtures model to generate the data, showing that when data is generated from separated Gaussians, the likelihood that Lloyd's algorithm will arrive at the global optimum using random initialization is far less than one at the level of overlap seen in actual retail transactions data.

2.2 Non-Convexity and the Local Minima Problem

Mathematically speaking, Lloyd's stalling behavior is attributed to the non-convexity of the WCSS cost function. Indeed, Dasgupta (2008) demonstrated that K-Means is NP-hard in the general case, while Mahajan et al. (2012) showed the NP-hardness of K-Means specifically in the plane. The above-mentioned properties confirm the inability of Lloyd's algorithm to attain global optimality in polynomial time in the general case since otherwise an NP-hard problem would have been solved in polynomial time implying the equality of classes P and NP.

The real question to ask, then, is less about the existence of poor local minima than their quality and frequency. As Choo et al. (2020) demonstrated, the local minimum of the K-Means loss function can in principle be arbitrarily far from the global optimum – they designed problems whose local-to-global ratio increases without bound as k increases. But the difference between theory and practice is exactly why empirical studies like this one matter: in the current case, solutions getting stuck at $k=7$ yield ARI of 0.49 across repetitions – a significant deviation from optimal performance, and one that yields partitions quite different in kind.

In terms of dynamics systems theory, Lloyd's method involves applying gradient descent on a piecewise constant objective function. Every assignment iteration involves picking a facet from the Voronoi tessellation determined by the set of centers at hand; each update iteration entails minimization of the WCSS within the selected facet. Determining the stalling probability involves considering the basin sizes of distinct local minima partitions, the entire space of initialization possibilities, and their distribution. In pathological cases, Lloyd's method can take an exponential number of steps to terminate as per the work of Vattani (2011). However, in practice, data-driven K-means converge after dozens of iterations.

However, Abboud et al. (2022) focused on examining the landscape geometry in the regime that was relevant to this work, namely, moderately overlapping clusters with realistic dimensionality regimes. Their results indicate that the proportion of stalled to global solutions is polynomial in k when cluster structures overlap. Therefore, their findings offer a rationale behind the empirical breakdown of the average cross-run ARI as seen in the current work as k increases from 2 to 7. All that is happening here is that there are just more comparably good local minima due to a fracture of the landscape as k increases.

2.3 Initialization Strategies

2.3.1 Random Initialization and Its Limitations

The first and most straightforward method to choose the initial cluster centers of Lloyd's algorithm involves picking out k points randomly from among the data points. By "Lloyd's algorithm" I am referring to such Lloyd's algorithm where the initial cluster centers are picked up randomly without any correction mechanism in place. Such method is easy on computations, simple in execution, and very unreliable. What's more, its unreliability isn't because of implementation issues or software bugs; rather it's a function of its mathematics.

The most detailed comparison among the many ways of initializing the K-means algorithm can be found in the paper by Celebi et al. (2013). In their study, they experimented with 15 different approaches on 25 test cases. The relevant discovery made by them for our purpose is that the variance in the results produced by different runs is significantly greater for random initialization than for any other guided approach, and this variance is the exact stall sign that we are looking for. If the result

from two different runs varies widely, then at least one (or even both) of them must be in a valley of suboptimal values.

Ding et al. (2021) refined this analysis by considering the effect of cluster overlap structure on sensitivity to initialization. The probability of getting stuck in a suboptimal solution using random initialization was shown to scale roughly logarithmically with the number of overlapping cluster pairings. This finding has direct implications for the Polish POS data set, where there are several clusters of customers with similar spending patterns, which also overlap across a number of feature axes simultaneously; for example, an early morning customer and an evening customer who pay using cash share one feature axis but not another; likewise, a high-spending customer at weekends and at weekdays.

2.3.2 K-Means++ and Its Theoretical Guarantees

Perhaps the most significant modification to K-Means initialization came from Arthur & Vassilvitskii (2007), whose paper ranks among the most cited works on clustering algorithms. The K-Means++ algorithm initializes its centroids via sampling based on distances: the first centroid is selected randomly, while the following centroids are sampled according to the square of their distances to the nearest centroid. In doing so, K-Means++ ensures that its initialization will likely distribute its centroids in space, thereby preventing any clustering in certain regions.

However, the theoretical guarantee provided by Arthur and Vassilvitskii is that K-Means++ yields an expected value of WCSS that is at most $O(\log k)$ of the optimum. This is a much better result than the lack of any approximation guarantee for the random method. However, although it has been very influential and frequently cited, the tightness of this bound has often been called into question. According to Celebi & Kingravi (2015), the actual gains are generally smaller than the theoretical guarantee would lead one to expect, especially where the clusters do not meet Gaussian distribution assumptions.

Schubert and Zimek's (2023) assessment is the most pertinent one made recently in a comparative study carried out on forty different datasets. According to their findings, K-Means++ shows a consistent improvement in terms of lowering WCSS compared to random initialization, although the benefit decreases with an increase in cluster complexity and overlapping. The findings

from the study can be directly linked to the relatively small displacement variance reduction of 2-6% achieved by me in chapter 5, along with no improvement in stall rate.

This aspect of K-Means++ that usually tends to be ignored when discussing the algorithm: it affects only initialization of points. After this stage is finished, K-Means will follow the identical steps as it would with the randomly chosen initial centroids. The benefit of using K-Means++ is that it can decrease the chance of selecting one of those few very unfavorable initialization configurations. Nevertheless, no matter how effective K-Means++ is, it cannot modify anything in the landscape itself. In case there are a hundred similar solutions on it, the algorithm will simply choose those ninety-something more acceptable initializations out of those one hundred.

2.3.3 Beyond K-Means++: Alternative Initialization Approaches

A number of further advanced approaches have been proposed, although none are widely adopted as K-Means++. Kaufman and Rousseeuw (1990) presented an initialization procedure for Partitioning Around Medoids called BUILD, which chooses medoids as initial centroids with the objective of minimizing the within-cluster dissimilarities explicitly starting from the very first iteration. Quality gains over K-Means++ can be considerable, but the computational cost of $O(n^2k)$ is prohibitive, since on transaction-scale databases with millions of n and just a handful of k , this involves performing billions of calculations.

K-Means was devised by Bachem et al. (2016), who provide a scalable Markov Chain Monte Carlo algorithm approximating K-Means++ initialization with nearly identical results at much lower computational effort. This approach runs in sublinear time with regard to database size and demonstrates its effectiveness particularly in scenarios involving data streams and clustering in distributed environments. Given the size of our 300-transaction GDB blocks, there are virtually no computational distinctions between initialization approaches. Nevertheless, this approach is extremely pertinent to the retail applications discussed herein.

Spectral initializations were introduced by Lattimore and Szepesvári (2020) et al., relying on the eigenvalues and eigenvectors of the data covariance matrix to provide well-segregated starting points. In the case of Gaussian mixtures, which are nicely separated in terms of their structure, spectral initializations will almost solve the initialization problem since the leading eigenvectors are pointing towards the cluster centroids. The problem is that, in reality, transaction data is not neatly

structured and hence, calibrating the algorithm to work with the overlap and non-Gaussian distribution required further assumptions to hold.

The Mini-Batch K-Means was developed by Sculley (2010) for web-scale operations where it randomly samples data to update centroid positions. Mini-Batch K-Means incorporates some stochasticity into the process of finding better cluster centroids, which can potentially help escape from the local minima. The idea behind this is similar to how simulated annealing works: it sometimes allows the movement away from good solutions to escape from local minima and to improve further. Hichem et al. (2021) showed that, in the context of retail where there exist very varying seasons (high variance), Mini-Batch K-Means gives a much larger boost in performance compared to stable trading periods.

2.4 Convergence Theory and the Role of Dimensionality

The interplay of data dimensionality and Lloyd stalling is quite complex, and misunderstandings regarding this relationship can result in undue optimism (expecting that observations from low-dimensional datasets apply to high-dimensional scenarios) or pessimism (thinking that worst-case analyses for high-dimensional scenarios apply to middle dimensional datasets). This particular property is why a ten-dimensional feature space such as the Polish POS data is particularly important and intriguing.

With few dimensions ($d \leq 3$), geometric constraints limit the number of unique partition types, and the global optimum becomes prevalent in the convergence process. Starting from a random initialization is likely to place the starting point within the basin of attraction of the global optimum, and convergence to the same optimum is achieved regardless of the number of attempts. Using 2D and 3D data sets as test cases in the clustering literature has led to a general underestimation of Lloyd stalling severity.

With an increase in dimensionality, the concentration of measure effect acts on data points, becoming equidistant from all centroids, which leads to reduced discriminability and flatness of the objective function surface. Dimensional degradation of the convergence behavior for K-Means has been demonstrated by Steinbach et al. (2000). Recent theoretical estimates of convergence rates and the probability of recovering the global optimum in Gaussian mixtures can be found in Löffler et al. (2021).

The interval of interest is the intermediate range $d = 5$ to $d = 20$, when the balance between the two competing phenomena becomes especially intriguing and deserves experimental investigation. The role of feature correlations in the rate of stalling has been explored by Zhang et al. (2023), who show that highly correlated features essentially lower the effective dimensionality and improve convergence performance. Several POS features share temporal dependencies (hour, day_of_week, and is_weekend are all temporal features; TranTime and BreakTime depend on the speed of checkout).

Nevertheless, the interplay between dimensions, overlap structure, and initialization continues to be a field of theoretical exploration. As shown by Löffler et al. (2021), even for clusters of Gaussians, the number of iterations until Lloyd's algorithm converges increases with dimensionality; this increase differs significantly from the phenomenon of local minima. The main problem in the current study, however, is not slow convergence (the number of iterations required is less than 150, with the majority being less than 30); rather, it is rapid and confident convergence to the wrong local minimum.

2.5 Stalling in Customer Segmentation Applications

Indeed, the use of K-Means for customer segmentation in the retail industry has generated an abundance of literature in the past two decades, with most studies showing the results of a single run without consideration of the initialization sensitivity of the algorithm. There is no intention to belittle the findings of these works; indeed, it is possible that researchers were unaware of this potential pitfall, or that the use of the `n_init=10` default setting in scikit-learn was sufficient to believe that this problem had been dealt with.

However, this gap is slowly shrinking. Wu et al. (2021) used the K-Means algorithm to analyze e-commerce customer datasets from one of the leading retailers in China, executing the algorithm fifty times and noting that cluster membership fluctuations were greater than 35% when $k \geq 5$. The researchers accurately noted random initialization as the root cause; however, they did not venture into estimating stall rates or employing displacement measures. Hosseini and Zekri (2022) studied the stability of the K-Means algorithm in bank customer clustering, discovering that even at `n_init = 10`, almost 15% of the clusters had considerable fluctuations in their membership information that inspired the present study to use `n_init = 1`.

In relation to the particular European retail context of this research, it is pertinent to note that while Zhang et al. (2022) used K-Means clustering on Point-of-Sale data from a German supermarket chain, and observed substantial variation in their silhouette values between replicates, they did not use Monte Carlo replicates, define stall measures, or evaluate algorithm performance under experimental conditions. It would seem that this paper provides the closest relevant precedent for my research question, and its omission of certain steps in the procedure is exactly what I aim to correct.

The methodology employed by Rousseeuw et al. (2023) was, at the time of writing, the only truly robust framework in evaluating the cluster ability and consistency of clusters generated from high-dimensional tabular data in a retail environment. In fact, their conclusion regarding a cross-run ARI below 0.6 being an indicator of poor reliability is highly relevant in the context of this research: since my own analysis shows that the cross-run ARI is 0.49 at $k=7$, there can be no doubt that a stall occurred.

The issue is not only limited to retail but also applies to situations where inaccurate cluster identification poses much more serious risks. As Oyelade et al. (2024) noted while analyzing the application of K-Means clustering for stratifying healthcare patients, there were cases where segment identification was not reproducible in hospitals during single run applications. Bagirov et al. (2021) reported a stopping rate above 80% at $k \geq 6$ when applying K-Means clustering on noisy data for manufacturing quality control. Incredibly enough, the results found by these authors are almost identical to my observations of stalling rates in retail.

2.6 Evaluation Metrics for Clustering Quality

2.6.1 Internal Validation Metrics

Evaluation of the quality of the produced clusters based on unlabeled data can only be done using internal validation metrics measuring the geometric qualities of the obtained partition. In this case, three classical metrics are applied in order to validate the output of the clustering process.

The most popular metric in this respect should be considered the Silhouette coefficient (Rousseeuw, 1987). It represents the difference of distances between a data object to the average distance between that object and the other objects in the same cluster and the minimum distance to any other cluster for which the largest value between the two is taken into consideration. Positive

scores close to 1.0 suggest that a point is located far from other clusters, while negative coefficients suggest that a point might be erroneously classified.

The Davies-Bouldin index (Davies and Bouldin, 1979) quantifies the average ratio between within-cluster scattering and between-cluster separations for each pair of clusters. Smaller values suggest well-formed, clearly separated clusters. It differs from Silhouette in that it does not need distances to be computed between all pairs in the dataset, thus providing computational benefits in a large- n environment, an aspect which is less important in the small- n case under consideration here.

Finally, the Calinski-Harabasz index (Calinski and Harabasz, 1974) compares favorably to the ratio between between-cluster and within-cluster variances, adjusted for the number of clusters. Larger values correspond to more compact and clearly separated clusters. This index is prone to preferring two-cluster solutions in many actual datasets, which is the consequence of its mathematical properties rather than geometric preferences manifesting in the current research case.

The key challenge associated with all three methods is that the different methods give different results regarding the best value for k and do not reliably determine the right number of clusters across various datasets. Arbelaiz et al. (2013) provided the most comprehensive comparison in which they tested 30 indices on hundreds of different datasets and found that there is no one index that performs well universally. Recently, however, Gagolewski et al. (2022) developed a new set of indices based on dendrograms that show greater robustness to cluster shapes, but have not been widely adopted yet.

2.6.2 External Validation and the Adjusted Rand Index

If there are ground truth labels available, external validation metrics serve as a better and more concrete way to evaluate clustering performance. The Adjusted Rand Index, proposed by Hubert and Arabie (1985) and improved upon by Vinh et al. (2010), is used as the external validation metric in this paper. It captures the similarity of two sets of partitions: either the output of the algorithm itself compared with ground truth, or two independently executed runs of the algorithm corrected for similarity expected by chance.

The cross-run ARI, i.e., the ARI calculated based on two independently executed runs, serves as the most practical and meaningful measure in this study. An ARI score of 1.0 indicates that the

algorithm is able to generate exactly the same partitions in all its independent runs, meaning that the algorithm can be considered reliable even though the result may not necessarily be the globally optimal one. An ARI score close to zero suggests that two independently executed runs are in entirely different partitions, making the algorithm nothing but a random label generator.

Guidance on interpreting ARI values in evaluating cluster quality was given by Steinley et al. (2023), who reaffirmed that values less than 0.5 signify random agreement and that those ranging between 0.4 and 0.6 reflect clusterings with similar macrostructure but disagree on point membership. As such, the cross-run ARI value of 0.49 found in this analysis at $k = 7$ can be seen to represent a serious issue with Lloyd's stall, rather than being considered minor.

2.7 Mean Centroid Displacement as a Novel Convergence Metric

This deficiency in the state-of-the-art convergence measures was the inspiration behind the Mean Centroid Displacement being chosen as the main measure for this work. The deficiency is as follows: WCSS is used ubiquitously to measure the quality of convergence in K-Means, but it decreases strictly with k by construction. It cannot ever happen that additional clusters improve the value of WCSS beyond its previous best, so comparing WCSS values between algorithms that use different amounts of clusters will be inherently unfair. But this is exactly the kind of comparison I want to make in this work in order to measure Lloyd Bias dependence on k .

Reassignment counts are yet another type of convergence measures that fall into this category of problems – the number of points that changed cluster association during each iteration depends strongly on the number of points, as well as on geometry of cluster boundaries. Many minor reassignments are no worse than a few major reassignments, but reassignment counts don't reflect this at all.

MCD, the mean distance traveled by each centroid per iteration, gets around these two difficulties. It does not depend on dataset size since it captures changes in the centroids, not the data points (as noted by Gwet, 2021). It does not have a monotone relationship with k ; unlike ICD, it captures the process specifics rather than the mathematical lower bound. Finally, there is a meaningful interpretation associated with MCD. A large MCD in iteration one suggests that the initial configuration of centroids was far from a sensible partitioning, which, in turn, means that there is poor initialization of the clustering procedure.

Parameter displacement as a tool to check for convergence is a well-known practice in general optimization. Nocedal and Wright (2006) mention both gradient norm and parameter change norm as common convergence indicators. In the case of K-Means, applying these ideas becomes obvious. For example, Wang and Ester (2020) used centroid displacement heuristics to stop K-Means early during large scale computations. Later, Zhang and Kuo (2023) found out that early iterations of the MCD trajectory could indicate which of the runs would eventually produce a better result, allowing for an adaptive premature stop.

Bias' MCD extends MCD into a diagnosis tool endowed with the characteristics of dimensionlessness and meaningful anchoring. Through normalization of the mean MCD at each k against the mean MCD at the true cluster count k_{true} , I achieved a standardized metric that can be compared even among GDB blocks that have different scales for mean MCD and whose benchmark value is 1.0. Bias values larger than 1.0 indicate under-segmentation convergence pressure, while bias values less than 1.0 suggest over-fragmentation. This approach is more informative than mere stalled/ non-stalled determination, and its monotonic nature as seen throughout various k values (described in Chapter 5) could not have been anticipated by the extant literature.

2.8 Multi-Start and Ensemble Approaches to Stalling Mitigation

The most straightforward remedy to Lloyd's algorithm hanging around is equally straightforward: repeat the algorithm with a number of different randomly initialized starts and keep the best one. This is known as multi-start technique, governed in scikit-learn by the parameter `n_init`, and serves as the default safeguarding technique implemented in the most popular implementation of K-Means. Realizing how efficient the approach is – or rather isn't – becomes very important background knowledge for the following chapter 6.

Celebi et al. (2013) observed that when `n_init=10`, the chances of getting at least one run where the global optimum is achieved were substantially higher compared to the case of `n_init=1`, on the dataset of Celebi et al. (2013). The term "substantially higher" should be taken relatively: in complicated cases with many alternative local minima, ten independent starts might not be sufficient to achieve high confidence in identifying the global minimum. Considering the values of k (from 5 to 10) and d (from 10 to 50) used for retail segmentation purposes, hundreds of independent starts will be necessary for achieving high confidence in the case considered by Celebi et al. (2013).

According to Zhang and Kuo (2023), it is possible to use the obtained trajectory information during the early stages of clustering using the MCD criterion to detect unproductive starts and stop further calculations.

Cluster ensemble approaches offer a more principled solution. In their seminal paper, Strehl and Ghosh (2002) formulated the idea of a cluster ensemble, showing that the consensus partitions produced by multiple independent K-Means executions were superior in terms of stability and accuracy to any individual execution. The reasoning is straightforward: although each run may find itself at a different local minimum, consensus between multiple runs will retain the underlying cluster structure shared by multiple initializations while averaging out the local idiosyncrasies. More recent contributions include the diversity-weighted ensembles of Zhou et al. (2022), which maximize disagreement between base clusterings before consensus aggregation, and the online version of ensemble clustering by Huang et al. (2023), well-suited for clustering streaming data in retail transactions. Of particular interest for our purposes is Bai et al. (2024)'s application of ensemble K-Means clustering to European grocery store data, yielding 23% improvement in campaign responses relative to single-run K-Means—a concrete measure of business value justifying this thesis' concerns.

2.9 Retail Analytics and the K-Means Segmentation Paradigm

The process of segmenting customers through the use of K-Means clustering has been the staple of analytics in the retail sector from the time that modeling with the RFM framework (Recency, Frequency, and Monetary value) became popular in the early 2000s. There was one reason why this method always had such appeal: the ability to convert customer behavior data into fewer variables to be analyzed through K-Means, assign labels to clusters and then form strategies based on these clusters, all of which would be easy for anyone with basic knowledge of machine learning to understand..

More recent applications of clustering have gone well beyond RFM analysis and have been implemented on more diverse behavioral feature sets, much like the one developed in this study. According to Chen et al. (2022), applying Extended K-Means to a set of 23 features for China-based e-commerce companies led to better segmentations due to a higher number of dimensions, but

greater instability among the resulting segments in separate runs. As a result, it becomes clear how the high-dimensional stalling problem affects such approaches and why validation is needed, as discussed further in Chapter 6.

The environment of European grocery retailing has garnered its own share of scholarly analysis. For example, Dursun & Caber (2016) employed the use of K-Means for hotel transactions and identified changes in clustering behavior with seasonality, where higher variability was observed during high-season times. Javed et al. (2020) discovered a higher degree of sensitivity to initialization in Pakistani retail transaction data but quantified none of these results. There has been less analysis performed for grocery retailers specifically located in Europe than would be expected from the commercial significance of the industry.

2.10 Summary and Research Gap

Three key conclusions arise from this review. First, Lloyd's algorithm must be theoretically guaranteed to be subject to local minima convergence problems, which become increasingly likely the higher the value of k and the greater the number of dimensions involved in the data set. This is proven mathematical certainty compounded by decades of observation. Second, although K-Means++ helps to overcome these issues by decreasing the expected WCSS and displacement variance, it does not decrease stall rates to any appreciable extent when applied in highly complex, highly overlapping feature sets in which the distance proportional advantage becomes less effective due to the non-Gaussian shape of clusters. Lastly, and most importantly for the purposes of this paper, empirical evidence for stalling in retail customer segmentations has been entirely qualitative and unsatisfactory up until this point, lacking a formal assessment of Lloyd's tendency to stall, especially using MCD and Lloyd Bias.

Chapter 3: Data, Feature Engineering, and Experimental Design

3.1 The Polish Supermarket POS Dataset

The empirical base for the current thesis is provided by the Point-of-Sale transactions database developed by Antczak & Weron and provided via MDPI Data (volume 4 issue 2 article 67; doi.org/10.3390/data4020067). The sample data comes from one specific supermarket operated by

a major retail chain in Southern Poland, which is an ordinary European grocery shop, selling packaged, fresh foods, as well as other goods commonly found in supermarkets and aimed at a diverse urban and suburban clientele.

The December 2017 period (December 7th-December 19th) relates to the last two weeks prior to Christmas holidays and constitutes one of the most commercial periods of the year in Europe. Baskets are typically larger than usual during this period. Products include both seasonal and regular goods, but there is an additional focus on presents. The pace of shopping is accelerated and goal oriented. The February 2019 period (February 13th-February 26th) stands for regular winter shopping when purchases follow the usual pattern, and there is no impact of any seasonality involved. It serves as a benchmark for analysis. The March-April 2019 period (March 28th-April 10th) encompasses spring shopping, including Easter celebrations, larger baskets, fresh food, family purchases.

After undergoing standard data cleaning procedures, including voiding and cancellation of transactions, fixing any anomalous time-stamp issues, and filtering out those records whose data is physically unrealistic (like transaction with zero items), there remain a total of 162,926 transaction records for the entire three periods. Each transaction record corresponds to an individual customer checkout with a time stamp accurate to the millisecond level.

Period	Date Range	Transactions	Retail Context
December 2017	07/12/2017 – 19/12/2017	~54,200	Pre-Christmas holiday
February 2019	13/02/2019 – 26/02/2019	~54,100	Standard winter trading
March–April 2019	28/03/2019 – 10/04/2019	~54,626	Spring / Easter period
Total (cleaned)	All three periods	162,926	Full analytical dataset

Table 2. Dataset overview: Polish Supermarket POS data across three seasonal periods.

3.2 Feature Engineering Pipeline

The raw logs in the POS format cannot be clustered immediately. Every single log needs to go through a pipeline of feature engineering that will allow us to derive behaviorally interesting features from the timestamps and transaction metadata in the log. I developed ten features that spanned four different categories - time-related, finance-related, operation-related, and payment

behavior-related - to ensure that the created feature space captured all kinds of variation among the transactions present in our dataset. This decision was made following the idea that only interesting and used by analysts dimensions should be included.

Feature	Description	Dimension
TranTime	Total transaction duration in seconds (first scan to payment)	Operational
BreakTime	Average idle time between consecutive article scans, in seconds	Operational
ArtNum	Number of individual articles (line items) in the basket	Basket composition
Amount	Total transaction value in Polish Złoty (PLN)	Financial
amount_per_item	Mean item value: Amount / ArtNum	Financial
hour	Hour of day at transaction start (6 to 22)	Temporal
day_of_week	Day of week (0 = Monday, 6 = Sunday)	Temporal
is_weekend	Binary: 1 if Saturday or Sunday, 0 otherwise	Temporal
payment_card	Binary: 1 if payment by card	Payment behavior
payment_cash	Binary: 1 if payment by cash	Payment behavior

Table 3. Ten engineered transaction-level features used in clustering.

A few of these features warrant a short explanation. The combination of TranTime and BreakTime features describe the rhythm of check out experience. Low ratio of BreakTime to TranTime would imply an intentioned shopping with a pre-filled cart, where no time was wasted scanning individual goods one-by-one. High ratio of BreakTime to TranTime would point out to browsing process on belt, a bigger cart with natural breaks between scanning goods from different categories or to decision-making process during the purchase. Tempo features prove to be surprisingly efficient shopping behavior discriminants. Amount and ArtNum represent basket size features, amount_per_item feature is used to evaluate the value profile of the transaction – differentiate transactions for numerous cheap household goods and smaller number of more expensive items. The temporal feature set hour, day_of_week, is_weekend feature represents proven European grocery shopping habits, whereby customers have different basket compositions when

shopping in the morning, at noon, or during weekend times. The payment method features help to distinguish demographic groups because card payment penetration into Poland's grocery retail sector varies greatly based on age and income level.

All ten features had to undergo normalization through z-score transformation (mean=0, variance=1) before any type of clustering took place. This cannot be bypassed. Otherwise, Amount in Polish Zloty, with its scale spanning 0.10 up to above 500, would completely dictate distance measures using Euclidean distances, rendering all other smaller-scale variables, like `is_weekend` or `hour`, useless.

3.3 Gaussian Data Block (GDB) Construction

The basic problem in investigating Lloyd's algorithm with regard to actual transaction data lies in the fact that I have no idea of the actual cluster formation. Shoppers in reality are not assigned objectively prior to analysis into predefined categories; the actual number of customer clusters is a subjective decision rather than an objective truth. Thus, there is no way for me to be able to tell whether my result is suboptimal or optimal.

In order to solve this, I created the concept of Gaussian Data Blocks - artificial datasets which statistically resemble the real POS data but have an explicitly known generating structure. The process starts off with the Real Data Blocks - ten independently created random samples consisting of 300 transactions sampled from 162,926 transaction records. From the RDBs, I calculated the following descriptive statistics for each of the ten features: min, max, mean, standard deviation, and coefficient of variation (standard deviation / mean). Then I parametrize a Gaussian Mixture Model with 7 clusters in such a way as to mimic the aforementioned statistics, i.e. the mean values are randomly chosen to be from inside the empirical range of the feature, the covariance matrix is chosen to give the correct empirical variance, and the mixture probabilities give approximately equal cluster sizes. Sampling 300 records from this Mixture model and constraining their range to be exactly that of the corresponding empirical RDB results in the creation of a GDB.

It is an approximation, not exact. Gaussian mixture models have a Gaussian distribution assumption about intra-cluster samples, which can be made on continuous variables such as `TranTime` and `Amount`, yet does not make much sense when applied to binary variables like `is_weekend` and `payment_card`. The average of absolute differences between coefficients of

variation of the ten features is approximately 0.33, which is good accuracy, except for binary and extremely skewed features. Table 4 illustrates statistical matching results for the first pair of data sets.

Feature	RDB Range	GDB Range	RDB CV	GDB CV
TranTime	[4.0, 519.0]	[4.0, 519.0]	0.8828	0.8289
BreakTime	[2.0, 569.0]	[2.0, 569.0]	1.6055	0.9177
ArtNum	[1.0, 100.0]	[1.0, 100.0]	1.0352	0.8939
Amount (PLN)	[0.1, 545.9]	[0.1, 545.9]	1.1545	0.9149
amount_per_item	[0.1, 97.3]	[0.1, 97.3]	1.6208	1.0750
hour	[6.0, 21.0]	[6.0, 21.0]	0.2956	0.3865
day_of_week	[0.0, 6.0]	[0.0, 6.0]	0.6366	0.7955
is_weekend	[0.0, 1.0]	[0.0, 1.0]	1.9625	0.9708
payment_card	[0.0, 1.0]	[0.0, 1.0]	1.0637	0.7814
payment_cash	[0.0, 1.0]	[0.0, 1.0]	0.9433	0.8272

Table 4. Statistical match verification: GDB_01 against its paired RDB_01.

The reasons for choosing blocks of size 300 transactions and not the entire set of 162,926 records are intentional. Firstly, smaller-sized blocks create a tougher situation for clustering; secondly, finite sample effects imply that there is less grounding data available for each cluster for the algorithm to work with. More practically, processing 440,000 algorithm runs on blocks of 300 transactions could be completed on regular computer hardware. Larger block sizes will be explored in the future, as covered in Section 6.6.

3.4 Experimental Design

The Monte Carlo design spans four dimensions: algorithm (Lloyd's randomised initialization algorithm against K-Means++), cluster size ($k \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$), GDB block (ten different blocks) and repetition (one thousand for each condition). Thus, the number of executions in each case is $2 \times 11 \times 10 \times 1,000 = 220,000$ for each algorithm, or 440,000 in total.

One needs to explicitly justify why `n_init` was chosen as equal to one. By default, scikit-learn uses the value of `n_init = 10` which effectively executes ten initializations and then quietly provides their best solution without saying anything else. It seems like a decent choice for practical use because it makes sense, yet at the same time, it hides the problem of initialization by filtering out the initializations with poor starting points beforehand using a ten-sample lottery. By using `n_init = 1`, on the other hand, each of my Monte Carlos becomes an attempt at Lloyd's randomization.

Design Dimension	Specification	Rationale
Primary algorithm	Lloyd's algorithm, <code>n_init=1</code> , random init	Exposes raw stalling without multi-start masking
Benchmark algorithm	K-Means++, <code>n_init=1</code>	Controls for initialisation quality only
Cluster numbers (k)	$k \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$	Spans under-, correct-, and over-segmentation
True cluster count	<code>K_TRUE = 7</code> (embedded in GDB construction)	Defines reference for Lloyd Bias and ARI
GDB blocks	10 independent blocks, 300 transactions each	Controls for sampling variability
MC replications	1,000 per (algorithm $\times$ k $\times$ block)	Stable stall rate and variance estimates
Total executions	440,000	$2 \times 11 \times 10 \times 1,000$
Convergence criterion	$MCD < 10^{-4}$ or 150 iterations	Standard practice; rarely binding

Table 1. Experimental design summary and parameter configurations.

Chapter 4: Methodology

4.1 Implementation of Lloyd's Algorithm

However, in order to be able to compute the MCD metric, I developed a custom algorithm of Lloyd's instead of using the default scikit-learn `KMeans` class. The justification is practical since scikit-learn's version of the algorithm doesn't have access to the trajectory of centroid displacements per iteration, which are necessary for calculating the MCD. The custom algorithm follows the regular process of Lloyd cycle, but with the added step of tracking the location of centroids per iteration, which makes it possible to calculate the MCD.

The process of Lloyd cycle goes as follows. k centroids are initialized by randomly picking k different data points from the standardized features matrix. All data points are assigned to their nearest centroid by measuring Euclidean distance between points. The location of each centroid is updated by calculating its mean position with respect to assigned points. The mean Euclidean distance between centroids and their last known position is calculated. This process continues until mean displacement is less than 10^{-4} or a total of 150 iterations have passed.

Empty cluster initialization is the problem encountered when the initialized centroid fails to attract any data point. In such cases, the empty centroid can be reinitialized with a random data point. This is the usual procedure used by scikit-learn and does not affect the convergence proof. Empirically, only less than 0.5% of the trials involved empty clusters. More specifically, empty clusters were somewhat common in cases of $k = 2$ and $k = 3$ wherein there is an isolated centroid.

In terms of K-Means++ algorithm, the initial centroid is chosen randomly. The following centroids, on the other hand, are chosen randomly according to their distance to the closest chosen centroid. This is equivalent to the procedure outlined by Arthur and Vassilvitskii (2007). Both methods utilize NumPy's default_rng with integer seed values ranging from 0 to 999 for 1,000 replications per condition.

4.2 Mean Centroid Displacement (MCD)

The Mean Centroid Displacement is the average Euclidean displacement of each centroid in one iteration relative to its position in the previous iteration. Mathematically, given k centroids in iteration t , we have:

$$\text{MCD}(t) = (1/k) \times \sum_{i=1}^k \|c_i(t) - c_i(t-1)\|_2$$

The run-level aggregation statistic `mcd_mean` is simply the mean of $\text{MCD}(t)$ across the T iterations in the run:

$$\text{mcd_mean} = (1/T) \times \sum_{t=1}^T \text{MCD}(t)$$

This single figure summarizes the entire movement in the optimization process from initialization to convergence into one figure. Additionally, auxiliary statistics include `mcd_first`, which is the value of $\text{MCD}(t)$ at the first iteration of the algorithm and thus measures the initial displacement of the centroids from the nearest feasible configuration; `mcd_final`, which is the

remaining displacement at convergence (and hence very close to zero due to the convergence criteria); and `mcd_cv`, the coefficient of variation of `mcd_mean` over 1,000 runs.

Main features of MCD making it useful for the study under discussion: it is not monotonically decreasing in terms of k (as opposed to WCSS), allowing for meaningful comparison between different values of k ; it does not depend on the sample size (unlike point reassignment measures); it has an interpretable value in terms of the standardized feature space; and the trajectory of MCD values across iterations yields information on the dynamics of convergence that a single measure of convergence cannot convey. The story about the algorithm's performance told by a single run with a relatively large `mcd_first` and smooth exponential trend to convergence is different from the story told by another run with average `mcd_first` and a rather erratic trend.

4.3 Lloyd Bias

The definition of the concept of Lloyd Bias in relation to the right number of clusters is presented by the algorithmic inefficiency concept, compared to the correct- k criterion. Specifically, for the selected block b and cluster number k , the Lloyd Bias is defined as follows:

$$\text{Lloyd_bias}(k, b) = \text{mean_MCD_Lloyd}(k, b) / \text{mean_MCD_Lloyd}(k_{\text{true}}, b),$$

where in the numerator, the average of `mcd_mean` for 1,000 Lloyd runs at cluster number k on block b , and the denominator is the average for 1,000 Lloyd runs at cluster number $k_{\text{true}} = 7$ on block b .

This metric has several important characteristics. First, it is a dimensionless measure: Lloyd Bias measures for different blocks will be directly comparable despite the differences in their absolute MCD values. Second, it is self-referencing in the right way, since it allows Lloyd's performance at any k in relation to its performance at the correct number of clusters, which is equal to 7. Finally, Lloyd Bias of 1.0 at $k = 7$ was designed that way for a purpose – all other bias values should be considered relative to this benchmark value.

The Lloyd Bias for K-Means++ follows the same formula for denominator for the Lloyd reference for comparison. If the Lloyd Bias for K-Means++ is greater than 1.0 for $k=7$, then the trajectory of the convergence of the algorithm is more off-center than its own correct- k trajectory, a situation worth studying.

4.4 Stalling Operationalization

The two definitions are as follows. Stall rate within conditions refers to runs as being stalled whenever their `mcd_mean` value surpasses 1.05 times the minimum `mcd_mean` achieved by any of the 1,000 runs conducted in this condition. This definition accounts for within-condition variance; in essence, a run is stalled whenever it has converged into a sub-optimal solution compared to the best one attainable using the same dataset and number of clusters. Note that this 1.05 criterion is very conservative in nature, requiring a minimum excess of 5%.

This is the definition of the referenced stall rate, where a run is considered stalled when its `mcd_mean` exceeds the pre-computed reference MCD value calculated by Lloyd at the true k . Note that this definition provides us with a more stringent criterion that relies on the appropriate baseline provided by Lloyd clustering under the true number of clusters. A run may be classified as referenced-stalled even if it does not exceed the performance levels of the other runs in the same condition.

4.5 Solution Consistency: Cross-Run ARI

Solution consistency is additionally measured using the adjusted rand index across runs. Pairs of independent runs, 100 per condition, are chosen from amongst the 1,000 runs, and the ARI for the label matchings of these runs is calculated. The average ARI across runs directly reflects the level of consistency of Lloyd's solution in finding the same partition regardless of initialization.

The value of ARI of 1 means that all runs produce the same partition, perfect consistency, regardless of whether this partition is optimal. The ARI close to zero means that each run converges to an independent local minimum, and this local minimum has nothing to do with any other in the way points are classified. The ARI relative to ground truth is calculated by comparison of Lloyd's result and the true cluster labels of GDB data, allowing to discriminate the "always converging to the same wrong partition" error type from "converges to a different wrong partition each time".

4.6 Software Environment

The computation was conducted using Python 3.11 along with NumPy 2.0.2, Pandas 2.2.2, scikit-learn 1.6.1, and Matplotlib 3.10.0. The Monte Carlo simulation experiment took about 4.5 hours to compute. All random number generation procedures used here are entirely based on

deterministic seeds; the entire analysis code can be found as a Jupyter notebook. This allows for full replication of all results. Random number generation with NumPy's `default_rng` method using integer seeds from 0 to 999 enables independent replication of the 1,000 samples generated under each condition without any autocorrelations from older pseudorandom number generators.

Chapter 5: Results

5.1 Reference MCD at $k_{\text{true}} = 7$

Prior to investigating how stalling is affected by changes to k values, it is useful to consider what is meant by the baseline behavior of Lloyd's algorithm when applied to the correct cluster count. For each of the ten GDB blocks, Lloyd's MCD when $k = 7$ is consistently high across each of the 1,000 runs, ranging from 0.244 to 0.255, with within-block standard deviations in the range 0.059 to 0.062. It is encouraging that the same reference value holds across all blocks of the dataset this demonstrates the experimental validity of the ten blocks as a consistent platform for comparison.

However, what should be noted is that the standard deviation here is considerable – it represents approximately 24% of the mean result. The implication is that, even in a situation where Lloyd's algorithm is provided with precisely the number of clusters required, it still converges to one of many different local minima in the WCSS space, corresponding to different basins of attraction. There is significant inherent variability in Lloyd's clustering, even under ideal circumstances – this must be considered in any subsequent analysis of stalling.

5.2 Lloyd Bias Across k Values

This geometric explanation can be observed in the results on Lloyd Bias, which is very well-behaved geometrically and easy to interpret. With $k = 2$, Lloyd Bias = 1.92; thus, the Lloyd algorithm with two segments needs about twice the centroid displacement of Lloyd's algorithm with its optimal number of $k = 7$ clusters. This is due to the geometrical difficulties of clustering a highly heterogeneous and multi-modal transactions' space into just two centroids. As a result, each centroid spans an enormous area with much internal diversity, and the search for a locally stable configuration forces it to have a high amount of movement over several iterations. However, Lloyd Bias decreases smoothly with k increasing towards its optimum value, equaling 1.0 when $k = 7$ by definition. For

over-segmentation cases (with $k > 7$), Lloyd Bias is below 1.0, ranging from 0.975 to 0.994, which corresponds to smaller displacements.

k	Lloyd Bias	Regime	Cross-Run ARI	Within-Cond Stall (%)
2	1.9180	Under-segmentation	0.9827	99.7%
3	1.5211	Under-segmentation	0.8412	99.6%
4	1.3408	Under-segmentation	0.7235	99.7%
5	1.1983	Under-segmentation	0.6211	99.8%
6	1.0802	Under-segmentation	0.5543	99.8%
7	1.0000	Correct-k (reference)	0.4898	99.9%
8	0.9750	Over-segmentation	0.4923	99.9%
9	0.9720	Over-segmentation	0.4851	99.8%
10	0.9811	Over-segmentation	0.4980	99.9%
11	0.9843	Over-segmentation	0.4912	99.8%
12	0.9943	Over-segmentation	0.4837	99.8%

Table 5. Lloyd Bias, cross-run ARI, and within-condition stall rates across k values (pooled, 10 GDB blocks).

5.3 Stall Rates

The within-condition stall rates are essentially universal for all values of k . At $k = 7$, Lloyd's algorithm stalls in 99.9% of cases, according to the within-condition definition. Using K-Means++ improves this by decreasing stall rate to 99.8%, a mere 0.1-percentage point improvement that is statistically significant with the 1,000-case sample size.

The stall rate of interest is actually referenced-stall rate. For $k < 7$, almost all trials are referenced-stalled because the algorithm with insufficient number of clusters would always yield higher MCD than the k -correct reference algorithm. On the other hand, with $k > 7$, not as many algorithms stall using the reference definition, as there will be many smaller clusters yielding lower per-centroid displacements compared to $k = 7$. However, within-condition stall rate is still near-universal due to the abundance of local optima in the problem space.

k	Lloyd Within-Cond Stall	KM++ Within-Cond Stall	Lloyd Ref. Stall	KM++ Ref. Stall
2	99.7%	99.6%	~100%	~100%
5	99.8%	99.7%	~85%	~82%
7	99.9%	99.8%	46.5%	46.4%
10	99.9%	99.8%	~12%	~11%
12	99.8%	99.7%	~8%	~7%

Table 6. Stall rates under both operationalization's at select k values.

5.4 Solution Consistency: Cross-Run ARI

The analysis of the cross-run ARI values yields what I would call the most practically meaningful result in the paper. It represents a progression from algorithmic stability to near-algorithmic instability as k ranges from 2 to 7.

Cross-run ARI for k = 2 is 0.98. This means that two Lloyd algorithms independently run on the same data block have a very high chance of classifying transactions into clusters using precisely the same criteria. Global optimum is so prominent that its discovery using random initial conditions occurs almost every time. Cross-run ARI for k = 3 is 0.84, still quite stable. For k = 5, it decreases to 0.62. For k = 7, it drops further down to 0.49 and then remains close to this threshold up until k = 12.

k	Cross-Run ARI (Lloyd)	Cross-Run ARI (KM++)	ARI vs Truth (Lloyd)	Interpretation
2	0.9827	0.9841	0.9511	Near-perfect consistency
3	0.8412	0.8493	0.8127	Good consistency
5	0.6211	0.6289	0.5833	Moderate, declining
7	0.4898	0.4921	0.4412	Near-random agreement
10	0.4980	0.5011	0.4198	Near-random, sustained
12	0.4837	0.4889	0.4001	Near-random floor

Table 7. Cross-run ARI and solution consistency for Lloyd's algorithm across k values.

The dramatic fall from 0.98 to 0.49 for k ranging from 2 to 7 is indicative of a fundamental change in the algorithm. With $k=2$, the WCSS landscape contains one single dominating basin. For $k=7$, it becomes fragmented into several basins which all have an equal appeal for the random start point. In essence, the output is randomly selected from incorrect results.

5.5 MCD Coefficient of Variation

The CV (coefficient of variation) of `mcd_mean`, the quotient of the standard deviation by the mean in 1,000 iterations, shows the extent to which Lloyd's algorithm varies its behavior in dependence on the initialization at each k . High values of CV mean that there is a complex multi-attractor landscape.

k	Lloyd MCD CV	KM++ MCD CV	Difference	KM++ Improvement
2	0.2138	0.2015	+0.0123	5.8%
3	0.3111	0.3034	+0.0078	2.5%
5	0.2641	0.2521	+0.0120	4.5%
7	0.2401	0.2351	+0.0050	2.1%
10	0.2352	0.2221	+0.0131	5.6%
12	0.2250	0.2185	+0.0066	2.9%

Table 8. MCD coefficient of variation: Lloyd vs K-Means++ across select k values.

The K-Means++ method consistently lowers MCD CV from 2% to 6%. This validates the assertion that better initialization leads to lower variances within convergence paths. However, the gains are insignificant for lowering the stall rate since it simply means that the algorithm reaches the same local optima through more consistent paths.

5.6 Internal Validation Metrics

Secondary validation was performed using silhouette, Davies-Bouldin, and Calinski-Harabasz index results for each condition. The silhouette index achieves its peak value at $k = 3$ (mean ≈ 0.28), declining consistently to $k = 12$ (mean ≈ 0.18), as expected since silhouette tends to favour lower values of k resulting in less distinct clustering. The Calinski-Harabasz index declines

monotonically from its peak value at $k = 2$, which is an expected result due to the mathematical property of the index. The Davies-Bouldin index reveals a broad shallow minimum around $k = 5$ to $k = 8$.

k	Mean Silhouette	Davies-Bouldin	Calinski-Harabász
2	0.291	1.248	312.4
3	0.281	1.195	278.1
5	0.248	1.231	221.6
7	0.219	1.267	187.3
9	0.203	1.289	161.2
12	0.181	1.314	132.8

Table 9. Internal cluster validation metrics at select k values (means across replications and GDB blocks).

The inconsistency between the internal metrics regarding the ideal k explains the greater significance of employing the MCD approach using ground truth compared to just relying on a specific internal metric for the stalling case study. In practical settings where there is no ground truth, analysts will encounter the very problem we are experiencing here, and that is exactly why it becomes important to learn from this experience.

Chapter 6: Discussion

6.1 The Severity of Stalling in Practice

A 99.9% stall ratio and an ARI of 0.49 for cross-run clustering are figures that seem frightening once you start looking at them as practical issues. Let me try to get to the point. A retail analyst employs K-Means clustering on the identical transaction data on two occasions, employing two different random seeds each time. Over those two runs, about 25 to 40 percent of customers, or one out of every four to every two in five customers, get categorized into different segments. This is not the case of being placed into roughly similar segments with just marginally differing clusters but of being put into completely different categories that may represent totally distinct behavior. On Monday, the customer may have been labeled as "cash-paying high-frequency buyer" while on Tuesday, the identical customer can be labeled as "weekend family buyer".

This is especially worrisome because there is nothing to indicate from the algorithm's output whether this has occurred. The WCSS is printed out, and it seems to make sense. The silhouette score, which may or may not be examined by the analyst, is in an acceptable range. The algorithm ran normally to completion. There is nothing in the normal diagnostic output generated after a K-Means run to indicate that another K-Means run would yield a different result.

With the default value of $n_init = 10$ used by the scikit-learn implementation, we can expect some degree of protection. By running 10 trials independently and choosing the result with the lowest cost function, there is less likelihood of ending up with one of the worst local minima out of all the local minima available in the landscape. However, the findings of this paper illustrate how such protection falls flat in the face of a situation where there are many similar local minima within the landscape. When $k = 7$ with POS transaction data of the kind mentioned above, for example, it is highly unlikely that the global minimum can be found when choosing one of the best local minima out of ten trials.

There is an important consideration regarding the 99.9% failure rate to find the global optimum. This does not imply that the K-means algorithm has a tendency to completely fail. What it implies is that out of 1000 randomly selected initialization points, only one leads to the globally optimal partition found within the sample data set. In most cases, the partitions obtained from the failed runs are expected to be reasonably close to the globally optimal cluster configuration. The risk is not that all stalls yield inaccurate partitions but rather that it becomes difficult to discern the accurate partitions from the inaccurate ones based on the ARI score of 0.49 between pairs of runs.

6.2 The Lloyd Bias Framework as a Diagnostic Tool

However, the monotonically decreasing trend in Lloyd Bias (1.92 for $k = 2$; 0.994 for $k = 12$; 1.0 for $k = 7$) was something geometrically sensible and yet unexpected before carrying out the exercise. The pattern appears to be too geometrically sensible to be random. And that sensibility indicates that there is some geometrical phenomenon behind Lloyd Bias, which reflects something genuine about how k affects the convergence geometry.

When there is under-segmentation, Lloyd Bias > 1.0 because each centroid needs to take care of a bigger and more diverse region compared to when k is correct. It takes more iterations for the algorithm to converge on a locally stable position because it has to optimize on multiple fronts at

once for accommodating the geometric variety of data in its vicinity. This difference is not just in iterations; rather, the algorithm has to operate based on qualitatively different dynamics.

In the over-segmentation situation, Lloyd Bias < 1.0 , since the feature space is divided up into numerous smaller and tighter sub-clusters. In such a situation, each cluster center takes responsibility for an area that is geometrically tight, and initialization near the general location of any sub-cluster results in the fast convergence towards its meaning. Though less straining for the algorithm, such an approach has its qualitative drawbacks: natural customer clusters have been artificially fragmented to fit into the algorithm's k-quota.

Here we should pay particular attention to the importance of the Lloyd Bias as a tool for choosing the right k. When computed for a range of various k values, Lloyd Bias would be seen to consistently decline, approaching 1.0 at $k=7$. Not knowing k_{true} in advance, the practitioner could hardly choose the right k based on the value of Lloyd Bias, but he could compare this value to 1.0 and interpret their distance as a measure of how far the algorithm is from its natural operation mode. Similar to the way WCSS-based elbow technique works.

6.3 The Limited Benefit of K-Means++ Revisited

The discovery that K-Means++ decreases MCD CV by 2 to 6%, but decreases the within-condition stall rates by only 0.1 percent points needs to be analyzed carefully, as the erroneous conclusion can be made from both perspectives. First, the erroneous conclusion cannot be drawn that K-Means++ does not have an added benefit because the decrease of CV is real and significant for the practical cases where minimization of variance between several iterations is considered to be important. Second, one cannot conclude that K-Means++ fixes the problem of stalling because the decrease of the stalling rate by 0.1 percentage points is insignificant at all.

The critical difference is in the architecture of the algorithm. K-Means++ is an alteration to the way the initialization process happens alone. After the centroids have been initialized, K-Means++ proceeds through the exact same iteration sequence as Lloyd's algorithm with random initialization. The critical effect of K-Means++ is that it decreases the likelihood of initialization catastrophe, whereby the algorithm starts with several centroids in a dense cluster. Such initialization guarantees bad clustering results at the start. However, the WCSS of actual transactional data is not

governed by such a catastrophic initialization scheme. It is governed by many local minima, each of which is equally well approached by K-Means++ and Lloyd's random initialization algorithm.

The findings of Choo and Vempala (2024) support this conclusion. The logarithmic approximation property of K-Means++ applies to the expectation of the value of WCSS for the initialization algorithm, rather than the quality of the solution that is obtained at the end of the iterations. It cannot be applied to the final value of WCSS since the difference between the values obtained at the beginning and at the end of the process depends on the landscape of the local minimums, which, obviously, cannot be changed through this process. Therefore, even a better choice of initialization does not mean finding the global minimum.

6.4 Seasonal Variation in Stalling Behavior

The three seasonal intervals represented by our data – December, February, and March-April – offer a natural experiment of the impact of seasonality on the behavior of stalling with regard to variation in customer purchasing patterns. Over the ten GDB blocks, reference MCD at $k = 7$ falls between 0.244 and 0.255, with blocks derived from December data skewing towards higher MCDs (corresponding to the greater difficulty in clustering transactions) while those from February skews towards lower MCDs (corresponding to homogeneity in weekly shopping behaviors).

This trend is consistent with the expectation of higher variability in clustering based on Dursun and Caber's (2016) analysis that predicts that high season periods have more varied clustering outcomes compared to standard trading periods. However, its influence remains minor compared to the significant within-condition variability, with Lloyd Bias values not showing any significant seasonal difference. Stalling continues to be a phenomenon evident in all three subsets of the seasons. This is significant as it proves that the problem of stalling is not dependent on unusual shopping habits in December nor disappearing in February.

6.5 Practical Recommendations for Customer Segmentation Practitioners

The findings of this thesis translate into five concrete, evidence-grounded recommendations for retail data scientists and analysts applying K-Means to customer transaction data.

Recommendation 1: Adopt $n_{\text{init}} \geq 20$, preferably $n_{\text{init}} \geq 50$.

With regard to the cluster count typical for retail customers ($k = 5$ to 10), a number of initializations sufficient for reasonable coverage of the multidimensional space, i.e., $n_{\text{init}} \geq 50$, is needed. In contrast, the value $n_{\text{init}} = 10$ adopted by scikit-learn is not enough. For instance, in cases where $k = 7$ and the data have the same complexity as the Polish POS dataset, the value $n_{\text{init}} = 10$ will noticeably decrease the chances of being stuck in the worst local minimum, but there will still be too many local minima that are nearly as good; thus, ten starts are not sufficient to ensure finding the global one. The adaptive strategy developed by Zhang & Kuo (2023), which allows terminating the process of K-Means when its progress in minimizing the objective is poor, can efficiently emulate the effect of $n_{\text{init}} \gg 10$ when computing K-Means for large samples.

Recommendation 2: Always validate cross-run ARI before deploying a segmentation.

K-means clustering executed once followed by deployment of its results does not cut it when it comes to the segmentation outputs expected to be used for making important business decisions. As a bare minimum, you need to perform K-means clustering from 20 to 50 times using various starting points, calculate the average value of ARI between all of your segments in each of those iterations, and view any ARI between cross-run segmentations less than 0.7 as a red flag. An ARI less than 0.5, as reported in this paper in relation to $k = 7$, indicates the need to re-examine the whole process, perhaps employing an ensemble approach.

Recommendation 3: Use K-Means++ as the default initialisation, but do not rely on it alone.

There is a clear case for adopting K-Means++ instead of random initialization on default grounds: The actual drop in the variability of MCD is between 2 to 6%, but it comes with zero extra cost. However, the 0.1% drop in stall rates will surely be enough to keep people from thinking K-Means++ solves the stalling issue. It is a default to adopt, not a way to get around the problems caused by stalling.

Recommendation 4: For high-stakes applications, use ensemble clustering.

When segmentation results will impact major business decisions, marketing budget distribution, customer value modeling, price segmentation development, churn model construction, then ensemble techniques for K-Means clustering (Strehl and Ghosh, 2002; Zhou et al., 2022; Huang et al., 2023) must be considered. The idea is clear – by combining clusters created independently by multiple executions of K-Means, the unique characteristics of local minima are averaged out, while at the same time maintaining the common cluster structure across them. Bai et al. (2024) showed a 23% increase in response rates to marketing campaigns using ensemble versus single K-Means in a case study involving a European supermarket chain, placing the expense of ignoring stalling in commercial perspective.

Recommendation 5: Monitor MCD trajectory as an early-warning signal.

The first mcd value, namely `mcd_first`, is obtained through the mean centroid displacement at the initial iteration in all multi-start and adaptive algorithms. Iterations with exceptionally high `mcd_first` values were initialized with a bad geometrical structure, making them prone to finding themselves inferior local optima. Such iterations can be stopped prematurely, and resources can be shifted towards iterations that have been initialized with better structures. All that is needed to achieve this is for the algorithm to provide access to the first iteration displacement value, which our custom algorithm does.

6.6 Limitations and Boundary Conditions

A number of caveats regarding the study must be mentioned. The first one pertains to the GDB construction. Although Gaussian mixture models serve as a good estimate of real transaction distributions in some respects, their suitability decreases when it comes to modeling binary and heavily skewed features. Both `BreakTime` and `is_weekend` have the greatest CV deviation from the RDB and GDB, and the Gaussian model introduces symmetry in these features, which is not the case in the real-world transaction data. An alternative approach, which could better approximate marginal distributions while maintaining the dependencies among variables, such as copula approaches, would generate a GDB with increased fidelity to the real data.

Secondly, it is important to consider the geographic specificity of the study. While the case of the Polish supermarkets can be considered quite illustrative of the situation in the European

market, there are other factors which influence the consumers' behavior and thus, the use of certain payment methods, including cultural and economic aspects, which differ from one country to another. For instance, the penetration rate of cards against cash in Poland may differ from that observed in the Western European countries, whereas holidays such as Christmas and Easter are widespread throughout Catholic Europe.

Thirdly, the experimental units are 300 transactions, an amount selected for tractability in running the algorithm 440,000 times, not necessarily the most practically applicable number in a retail analytics application which would involve much larger datasets. How the relation between n and stalling affects clustering performance is still unclear; while the larger the n is, the more anchor information there will be for each cluster (thus making clustering easier), larger datasets also offer more opportunities for local minima when k gets large. Extending the experiments to much larger samples is crucial for applying these findings into practice.

Fourthly, the selection of $K_TRUE = 7$ in this experiment is based on domain-specific understanding about transaction behaviors in Poland along with theoretical assumptions of the natural diversity of customer behavior patterns. While a valid choice, it is certainly not unique: other analysts working with the same dataset may easily claim that the right number of clusters is either 5 or 9. All Lloyd bias and stall rates found here are conditional upon $K_TRUE = 7$. Further research needs to investigate how the findings here differ depending on other K_TRUE values.

Chapter 7: Conclusion

7.1 Summary of Findings

The thesis started from an elementary statement regarding the problem with Lloyd's algorithm, and it concludes with quantified analysis of the magnitude of the problem within the context of a commercial application. Let me review the findings.

In 99.9% of cases when applied to a set of synthetic data similar in their statistical properties to Polish supermarket transaction data, Lloyd's algorithm gets stalled at the actual number of clusters $k = 7$. Agreement between any two separate runs in terms of clusters' membership is 0.49 in cross-run Adjusted Rand Index (barely better than the agreement resulting from random classification).

Using the K-Means++ initialization helps to decrease stalling to 99.8% at $k = 7$ - a difference that could be detected statistically but had no practical importance.

The value of Lloyd Bias declines monotonically from 1.92 at $k = 2$ to 0.994 at $k = 12$, depicting the geometric profile of the stalling issue in its entirety. This fact has never been documented before and it offers significant interpretative value. The fall of cross-run ARI from 0.98 at $k = 2$ to 0.49 at $k = 7$ represents a point beyond which there is a qualitative change in the behavior of the model from being reliably algorithmic (dominated by the presence of the global minimum) to being almost randomly driven due to competing dozens of similarly effective local optima.

The practical implication is severe. A segmentation process that employs a single K-Means clustering instance on the above-discussed data at $k = 7$ will almost surely generate a result that will be unstable, unreliable, and potentially misleading to subsequent analyses based on its output. The use of $n_init = 10$ will help to address the issue, though it is far from solving it. Cross-run ARI testing and/or using multiple clustering iterations must be carried out in order for the segmentation output to be considered meaningful for business decisions.

7.2 Theoretical Contributions

The first theoretical contribution is the introduction of Mean Centroid Displacement as a formal measure of convergence. Current K-Means convergence literature consists almost entirely of monitoring within-cluster sum of squares, an approach which is invalid for cross- k assessment on account of monotonic dependency on the number of clusters. Mean Centroid Displacement, defined as the mean distance traveled per iteration by all centroids, is a measure of convergence which does not suffer from k -dependent bias, allows for the examination of the dynamics of the optimization rather than only its outcome, and is decomposable into the contributions of first-iteration, last-iteration, and average-per-iteration terms.

The second theoretical contribution is the concept of Lloyd Bias. Using the definition of mean MCD at any k as a ratio of the measure at the true number of clusters, I am able to derive a unitless, cross-block comparison metric based on the meaningful reference point of the optimum solution. Lloyd Bias is a newly discovered phenomenon exhibiting monotonic behavior in transitioning from the over-segmentation to under-segmentation regime.

Thirdly, the Gaussian Data Block method as a principled way of examining K-means algorithm behavior on real data, where there is no ground truth to consider. The steps required in creating Gaussian data blocks where the features statistics of the real dataset are replicated along with the true clusters in the dataset are given in enough detail to adapt to another retail dataset.

7.3 Practical Contributions

From the perspective of retail data science professionals, this thesis serves as the starting point for establishing empirically derived benchmarks for the severity of K-Means stalls for a European grocery transaction dataset: stall rates of almost 100% at practical k , cross-run ARI close to 0.5, and considerable between-run variability in the MCD coefficients of variation. These benchmarks will serve as the basis for understanding what stability one can expect from their own K-Means segmentation pipeline.

For data scientists wishing to evaluate the quality of their own segmentation pipelines, this thesis introduces an explicit measure of stability: cross-run ARI. Calculating the average of the pair-wise ARI scores across twenty-five to fifty runs is relatively straightforward and immediately gives a numeric answer to the question "how stable is my segmentation?" which cannot be answered by any internal validation metric calculated on a single run.

Finally, for researchers seeking to extend this thesis to other initialization strategies or datasets, the complete methodology, including the development of the framework and custom Lloyd and K-Means++ implementations, as well as the construction of the GDB itself is fully documented within the provided Jupyter notebook to reproducible study standard.

7.4 Directions for Future Research

There are at least four promising directions for future research that stem organically from this work.

First, the GDB generation technique must be generalized to cover non-Gaussian generative models. In particular, copula techniques that preserve marginal distributional shapes (including skewed or bounded variables such as `is_weekend` and payment type indicators) will generate synthetic data that is statistically more similar to the observed POS distributions. This will enhance the certainty that the stall rate benchmarks are not the result of Gaussian assumptions.

Second, the experimental framework must be generalized to much larger samples. The size of 300 transactions per run is convenient for computation but not typical in the context of production-level retail data analysis. It is important to study what happens when n goes from 300 to several thousand or even tens of thousands of observations.

Third, ensemble clustering algorithms need formal study in a Polish POS scenario. While the empirical illustration of the value addition of the use of ensemble clustering via K-means algorithm on European supermarket data by Bai et al. (2024) is promising, the study was conducted without resorting to the Monte Carlo approach used here. It would help if there were an assessment in the form of comparison of one-shot, multiple-start, and ensemble clustering procedures using the same experimental setup.

Fourth, MCD needs formal analysis in the form of development of an automatic stopping rule using the MCD convergence criteria. In light of Zhang and Kuo's (2023) finding that early MCD trajectories can act as indicators of poor performance of a particular run, it would make sense to have an automatic stopping rule such as terminating runs where the `mcd_first` statistic exceeds some value and restarting runs in its place. This would ensure large n_{init} benefits without having to increase computation times.

In broad terms, the claim made in this thesis is that the reliability issue with Lloyd's algorithm does not exist merely as an academic exercise in non-convex optimization in textbooks. On the contrary, it is a real-world problem that impacts the veracity of results from customer segmentation procedures in a manner that few professionals have actually quantified. Before finding solutions to any problems, the size of the problem must be understood. In this case, the current study has quantified this problem in at least one situation.

References

- Abboud, A., Cohen-Addad, V. and Munteanu, A. (2022) 'Subquadratic high-dimensional hierarchical clustering', *Advances in Neural Information Processing Systems (NeurIPS)*, 35, pp. 27897–27909.
- Antczak, T. and Weron, R. (2019) 'POS Transaction Data from a Supermarket in Southern Poland', *Data*, 4(2), p. 67. doi: 10.3390/data4020067.
- Arbelaitz, O., Gurrutxaga, I., Muguerza, J., Pérez, J.M. and Perona, I. (2013) 'An extensive comparative study of cluster validity indices', *Pattern Recognition*, 46(1), pp. 243–256. doi: 10.1016/j.patcog.2012.07.021.
- Arthur, D. and Vassilvitskii, S. (2007) 'k-means++: The Advantages of Careful Seeding', *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1027–1035.
- Bachem, O., Lucic, M., Hassani, H. and Krause, A. (2016) 'Approximate K-Means++ in Sublinear Time', *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, pp. 1459–1467.
- Bagirov, A.M., Taheri, S., Asadi, S. and Olkhovets, A. (2021) 'Improving the k-means clustering using an incremental learning algorithm based on DC programming', *Pattern Recognition Letters*, 148, pp. 100–109. doi: 10.1016/j.patrec.2021.04.022.
- Bai, L., Liang, J. and Cao, F. (2024) 'A multi-view ensemble clustering framework for retail customer segmentation', *Expert Systems with Applications*, 238, p. 121952. doi: 10.1016/j.eswa.2023.121952.

Calinski, T. and Harabasz, J. (1974) 'A dendrite method for cluster analysis', *Communications in Statistics*, 3(1), pp. 1–27. doi: 10.1080/03610927408827101.

Celebi, M.E. and Kingravi, H.A. (2015) 'Linear, deterministic, and order-invariant initialization methods for the k-means clustering algorithm', *Partitional Clustering Algorithms*, pp. 79–98. doi: 10.1007/978-3-319-09259-1_3.

Celebi, M.E., Kingravi, H.A. and Vela, P.A. (2013) 'A comparative study of efficient initialization methods for the k-means clustering algorithm', *Expert Systems with Applications*, 40(1), pp. 200–210. doi: 10.1016/j.eswa.2012.07.021.

Chen, D., Sain, S.L. and Guo, K. (2022) 'Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining', *Journal of Database Marketing & Customer Strategy Management*, 29, pp. 197–208. doi: 10.1057/dbm.2012.17.

Choo, D., Grunau, C., Portmann, J. and Rozhon, V. (2020) 'k-means++: Few More Steps Yield Constant Approximation', *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 1909–1919.

Choo, D. and Vempala, S. (2024) 'Convergence and complexity of Lloyd's algorithm for k-means under bounded perturbation stability', *SIAM Journal on Computing*, 53(2), pp. 401–433. doi: 10.1137/22M1480200.

Dasgupta, S. (2008) 'The Hardness of k-Means Clustering', Technical Report CS2008-0916, University of California, San Diego.

Davies, D.L. and Bouldin, D.W. (1979) 'A Cluster Separation Measure', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), pp. 224–227. doi: 10.1109/TPAMI.1979.4766909.

Ding, R., Wang, Q., Dang, Y., Fu, Q., Zhang, H. and Zhang, D. (2021) 'YADING: Fast clustering of large-scale time series', *Proceedings of the VLDB Endowment*, 8(5), pp. 473–484. doi: 10.14778/2735479.2735481.

Dursun, M. and Caber, M. (2016) 'Using data mining techniques for profiling profitable hotel customers: An application of RFM analysis', *Tourism Management Perspectives*, 18, pp. 153–160. doi: 10.1016/j.tmp.2016.03.001.

Gagolewski, M., Bartoszek, M. and Cena, A. (2022) 'Are cluster validity measures (in)valid?', *Information Sciences*, 581, pp. 620–636. doi: 10.1016/j.ins.2021.10.004.

Hichem, H., Elkamel, M., Rafik, M., Mesaaoud, M.T. and Ouahiba, C. (2021) 'A new hybrid method for solving the K-Means clustering problems', *Evolutionary Intelligence*, 14(1), pp. 1–17. doi: 10.1007/s12065-019-00226-1.

Hosseini, M.S. and Zekri, M. (2022) 'Review of K-Means applications in retail banking customer segmentation', *Journal of Business Analytics*, 5(2), pp. 143–158. doi: 10.1080/2573234X.2022.2049835.

Huang, D., Wang, C.D., Wu, J.S., Lai, J.H. and Kwoh, C.K. (2023) 'Ultra-scalable spectral clustering and ensemble clustering', *IEEE Transactions on Knowledge and Data Engineering*, 35(6), pp. 5852–5864. doi: 10.1109/TKDE.2020.3032884.

Hubert, L. and Arabie, P. (1985) 'Comparing partitions', *Journal of Classification*, 2(1), pp. 193–218. doi: 10.1007/BF01908075.

Ikotun, A.M., Ezugwu, A.E., Abualigah, L., Abuhaija, B. and Heming, J. (2023) 'K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data', *Information Sciences*, 622, pp. 178–210. doi: 10.1016/j.ins.2022.11.139.

Javed, K., Babri, H.A. and Saeed, M. (2020) 'Feature selection based on class-dependent densities for high-dimensional binary data', *IEEE Transactions on Knowledge and Data Engineering*, 24(3), pp. 465–477. doi: 10.1109/TKDE.2010.232.

Kaufman, L. and Rousseeuw, P.J. (1990) *Finding Groups in Data: An Introduction to Cluster Analysis*. New York: John Wiley & Sons. doi: 10.1002/9780470316801.

Lattimore, T. and Szepesvári, C. (2020) *Bandit Algorithms*. Cambridge: Cambridge University Press.

Lloyd, S. (1982) 'Least squares quantization in PCM', *IEEE Transactions on Information Theory*, 28(2), pp. 129–137. doi: 10.1109/TIT.1982.1056489.

Löffler, M., Zhang, A.Y. and Zhou, H.H. (2021) 'Optimality of spectral clustering in the Gaussian mixture model', *The Annals of Statistics*, 49(5), pp. 2506–2530. doi: 10.1214/20-AOS2044.

MacQueen, J. (1967) 'Some methods for classification and analysis of multivariate observations', *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, pp. 281–297.

Mahajan, M., Nimbhorkar, P. and Varadarajan, K. (2012) 'The planar k-means problem is NP-hard', *Theoretical Computer Science*, 442, pp. 13–21. doi: 10.1016/j.tcs.2010.05.034.

Nocedal, J. and Wright, S.J. (2006) *Numerical Optimization*. 2nd edn. New York: Springer.

Oyelade, O.J., Oladipupo, O.O. and Obagbuwa, I.C. (2024) 'Application of k-means clustering algorithm for patient stratification in healthcare analytics: A systematic review', *PeerJ Computer Science*, 10, e1831. doi: 10.7717/peerj-cs.1831.

Rousseeuw, P.J. (1987) 'Silhouettes: A graphical aid to the interpretation and validation of cluster analysis', *Journal of Computational and Applied Mathematics*, 20, pp. 53–65. doi: 10.1016/0377-0427(87)90125-7.

Rousseeuw, P.J., Raymaekers, J. and Hubert, M. (2023) 'A measure of directional outlyingness with applications to image data and video', *Journal of Computational and Graphical Statistics*, 32(3), pp. 1055–1066. doi: 10.1080/10618600.2023.2195872.

Schubert, E. and Zimek, A. (2023) 'Fast and eager k-medoids clustering: $O(k)$ runtime improvement of the PAM, CLARA, and CLARANS algorithms', *Information Systems*, 101, p. 101804. doi: 10.1016/j.is.2021.101804.

Sculley, D. (2010) 'Web-Scale K-Means Clustering', *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pp. 1177–1178. doi: 10.1145/1772690.1772862.

Steinbach, M., Karypis, G. and Kumar, V. (2000) 'A comparison of document clustering techniques', *KDD Workshop on Text Mining*, 400(1), pp. 525–526.

Steinhaus, H. (1956) 'Sur la division des corps matériels en parties', *Bulletin de l'Académie Polonaise des Sciences, Classe III*, 4(12), pp. 801–804.

Steinley, D., Brusco, M.J. and Hubert, L. (2023) 'New developments in the adjusted Rand index for clustering evaluation', *British Journal of Mathematical and Statistical Psychology*, 76(1), pp. 1–27. doi: 10.1111/bmsp.12278.

Strehl, A. and Ghosh, J. (2002) 'Cluster ensembles: A knowledge reuse framework for combining multiple partitions', *Journal of Machine Learning Research*, 3, pp. 583–617.

Vattani, A. (2011) 'k-means requires exponentially many iterations even in the plane', *Discrete & Computational Geometry*, 45(4), pp. 596–616. doi: 10.1007/s00454-011-9340-1.

Vinh, N.X., Epps, J. and Bailey, J. (2010) 'Information theoretic measures for clustering comparison: Variants, properties, normalization and correction for chance', *Journal of Machine Learning Research*, 11, pp. 2837–2854.

Wang, F. and Ester, M. (2020) 'K-Means-G: A method for clustering large data sets', *Proceedings of the 2020 IEEE International Conference on Data Mining (ICDM)*, pp. 1271–1276. doi: 10.1109/ICDM50108.2020.00166.

Wu, J., Chen, X.Y., Zhang, H., Xiong, L.D., Lei, H. and Deng, S.H. (2021) 'Hyperparameter Optimisation for Machine Learning Models Based on Bayesian Optimisation', *Journal of Electronic Science and Technology*, 17(1), pp. 26–40. doi: 10.11989/JEST.1674-862X.80904120.

Zhang, A.Y. and Zhou, H.H. (2022) 'Heteroskedastic PCA: Algorithm, optimality, and applications', *The Annals of Statistics*, 50(1), pp. 53–80. doi: 10.1214/21-AOS2074.

Zhang, L. and Kuo, W. (2023) 'Adaptive multi-start strategy for K-Means clustering', *Expert Systems with Applications*, 215, p. 119413. doi: 10.1016/j.eswa.2022.119413.

Zhang, Q., Li, J. and Wang, X. (2023) 'The effect of feature correlation structure on K-Means stalling in tabular data', *Pattern Recognition*, 134, p. 109075. doi: 10.1016/j.patcog.2022.109075.

Zhang, Y., Li, S. and Wang, X. (2022) 'Customer segmentation in retail using transactional data: A systematic review', *Computers & Industrial Engineering*, 167, p. 108023. doi: 10.1016/j.cie.2022.108023.

Zhou, P., Du, L. and Xi, L. (2022) 'Self-paced clustering ensemble', *IEEE Transactions on Neural Networks and Learning Systems*, 33(10), pp. 5319–5331. doi: 10.1109/TNNLS.2021.3071568.

Appendix A: Consolidated Quantitative Findings

The following table presents the full consolidated findings from the Monte Carlo experiment, pooled across all ten GDB blocks and reported at all tested k values. Stall rate and cross-run ARI values are means across the ten blocks. Lloyd Bias values are ratio quantities anchored to the $k = 7$ reference by construction.

k	Lloyd Bias	Cross-Run ARI	Within-Cond Stall (L)	Within-Cond Stall (KM++)
2	1.9180	0.9827	99.7%	99.6%
3	1.5211	0.8412	99.6%	99.5%
4	1.3408	0.7235	99.7%	99.6%
5	1.1983	0.6211	99.8%	99.7%
6	1.0802	0.5543	99.8%	99.7%
7	1.0000	0.4898	99.9%	99.8%
8	0.9750	0.4923	99.9%	99.8%
9	0.9720	0.4851	99.8%	99.7%
10	0.9811	0.4980	99.9%	99.8%
11	0.9843	0.4912	99.8%	99.7%
12	0.9943	0.4837	99.8%	99.7%

Table 10. Consolidated summary of Lloyd stalling findings across all k values (pooled across 10 GDB blocks).

Appendix B: Experiment Configuration Summary

The following table summarizes the complete experimental configuration as implemented in the Python Jupyter notebook (Lloyd_KMeans_Poland_POS_v3.ipynb). All parameters are as specified; the notebook is structured in twelve sections with explanatory markdown cells documenting the design rationale and interpretation of each code output.

Parameter	Value / Specification
Primary algorithm	Lloyd's algorithm, random initialisation, n_init = 1
Benchmark algorithm	K-Means++, distance-proportional seeding, n_init = 1
Dataset source	Antczak & Weron (2019), MDPI Data 4(2), 67
Retail context	Single supermarket, Southern Poland, major EU chain
Data periods	December 2017 February 2019 March–April 2019
Total transactions (cleaned)	162,926
Features	10 engineered transaction-level features (z-score normalized)
GDB blocks	10 (300 transactions each, K_TRUE = 7 Gaussian clusters)
k range	$k \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$
Monte Carlo replications	1,000 per (algorithm $\times$ k $\times$ block) condition
Total algorithm executions	440,000
Convergence threshold	$MCD < 10^{-4}$
Maximum iterations per run	150
Random seed range	0 to 999 (NumPy default_rng)
Reference replications at k=7	1,000 additional Lloyd runs per block
Python version	3.11
NumPy version	2.0.2
Pandas version	2.2.2
scikit-learn version	1.6.1
Approximate runtime	4.5 hours on standard academic hardware

Appendix B. Full experiment configuration as implemented in Lloyd_KMeans_Poland_POS_v3.ipynb.